

Funnel logic:

View always happens first.

Click may happen after view (probabilistic).

After a click, there's a choice between save and cart (mutually exclusive in this simulation).

If the product is added to the cart, there's a chance the user buys it, updating product availability and logging a "buy" event with quantity and rating.

```
#load libraries

import random
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

```
#set seed
random.seed(42)
np.random.seed(42)
```

```
# load product dataset
import pandas as pd
product_df = pd.read_csv('cannabis-prod-data.csv')

# create map key = product_id, value = availability
product_avail_map = product_df.set_index('product_id')['availability'].to_dict()
```

```
#assuming we have the same product ids from our product dataset p001, p002, ... , p100
```

```
product_ids = list(product_avail_map.keys())
```

```
#generate 1000 user ids
user_ids = [f"u{i:04d}" for i in range(1, 1001)]
```

```
# possible funnel event types with probabilities

event_types = {
    "view": 1.0,
    "click": 0.70, #70% leads to clicks
    "save": 0.10, #10% from clicks results in save for later
    "cart": 0.50, #50% from clicks that aren't saved are added to the cart
    "buy": 0.80} #80% of carts lead to purchases
```

```
# ratings
```

```
rating_choices = [1, 2, 3, 4, 5]
rating_probs = [0.05, 0.15, 0.20, 0.30, 0.30]
```

```
# create a fct to generate a random timestamp within the last 90 days
```

```
def random_timestamp():
    start = datetime.now() - timedelta(days = 90)
    random_days = random.randint(0, 90)
    random_hours = random.randint(0, 23)
    random_minutes = random.randint(0, 59)
    random_seconds = random.randint(0, 59)
    return start + timedelta(days = random_days, hours = random_hours,
                             minutes = random_minutes, seconds = random_seconds)
```

```
# list to hold interaction rows
interactions = []
```

```
# simulate interactions
# events occur when random number is less than or equal to probab of event since
# random.random() makes number between 0 and 1

for user in user_ids:
    # number of products this user interacts with 5 to 20
    n_products = random.randint(5, 20)
    products_sample = random.sample(product_ids, n_products)

    for product in products_sample:
        # start at view
        timestamp = random_timestamp()
        interactions.append({
            "user_id": user,
            "product_id": product,
            "event_type": "view",
            "timestamp": timestamp,
            "rating": None,
            "quantity": None })

    #funnel stage 1: click
    if random.random() <= event_types["click"]:
        timestamp += timedelta(minutes = random.randint(0, 5)) #0-5 min from view to click
        interactions.append({
            "user_id": user,
            "product_id": product,
            "event_type": "click",
            "timestamp": timestamp,
            "rating": None,
            "quantity": None })

    #decide between save and cart
    if random.random() <= event_types["save"]:
        timestamp += timedelta(minutes = random.randint(1, 10)) #1-10min from click to save/cart
        interactions.append({
            "user_id": user,
            "product_id": product,
            "event_type": "click",
```

```

        "timestamp": timestamp,
        "rating": None,
        "quantity": None })

continue #stop here for save

elif random.random() <= event_types["cart"]:
    avail = product_avail_map.get(product, 0)
    if avail > 0:
        max_qty = min(avail, 10) #cap max cart qty at 10 or availability whichever is smaller
        quantity = random.randint(1, max_qty) #if more than 0, can only buy 1 to max_qty
    else:
        quantity = None

    timestamp += timedelta(minutes = random.randint(1, 10))
    interactions.append({
        "user_id": user,
        "product_id": product,
        "event_type": "cart",
        "timestamp": timestamp,
        "rating": None,
        "quantity": quantity })

# funnel step 3: buy
# only buy if quantity is true (false when none) and random.random() < p.event is true
if quantity and random.random() <= event_types["buy"]:
    timestamp += timedelta(minutes = random.randint(5, 20)) #5-20 from cart to buy
    product_avail_map[product] -= quantity #subtract from availability
    rating = random.choices(rating_choices,
                           weights = rating_probs, k = 1)[0]

    interactions.append({
        "user_id": user,
        "product_id": product,
        "event_type": "buy",
        "timestamp": timestamp,
        "rating": rating,
        "quantity": quantity })

```

```

#convert to df and sort by timestamp
df_interactions = pd.DataFrame(interactions)
df_interactions = df_interactions.sort_values("timestamp").reset_index(drop = True)

```

```

# show first 10 rows
print(df_interactions.head(10))

```

	user_id	product_id	event_type	timestamp	rating	quantity
0	u0318	p084	view	2025-05-17 00:46:51.853052	NaN	NaN
1	u0374	p072	view	2025-05-17 00:50:12.863955	NaN	NaN
2	u0711	p087	view	2025-05-17 01:01:42.930708	NaN	NaN
3	u0928	p087	view	2025-05-17 01:09:47.040863	NaN	NaN
4	u0852	p082	view	2025-05-17 01:10:59.959620	NaN	NaN
5	u0852	p082	click	2025-05-17 01:11:59.959620	NaN	NaN
6	u0986	p009	view	2025-05-17 01:14:30.100735	NaN	NaN
7	u0298	p058	view	2025-05-17 01:17:42.849450	NaN	NaN
8	u0298	p058	click	2025-05-17 01:22:42.849450	NaN	NaN
9	u0556	p061	click	2025-05-17 01:24:11.899913	NaN	NaN

```

# how many buy events
print(df_interactions[df_interactions["event_type"] == "buy"]["user_id"].count())

```

2604

```

# how many unique users had a buy event type
print(df_interactions[df_interactions["event_type"] == "buy"]["user_id"].nunique())

```

907

```
print(df_interactions[df_interactions["event_type"] == "buy"])
```

	user_id	product_id	event_type	timestamp	rating	\
14	u0298	p058	buy	2025-05-17 01:38:42.849450	2.0	
21	u0227	p058	buy	2025-05-17 01:59:08.835540	5.0	
28	u0251	p048	buy	2025-05-17 02:51:05.840370	3.0	
37	u0520	p026	buy	2025-05-17 03:21:50.892926	4.0	
42	u0636	p028	buy	2025-05-17 03:46:26.915791	3.0	
...	...	...	...	...	...	...
28963	u0627	p063	buy	2025-08-15 20:14:11.914110	2.0	
28968	u0856	p078	buy	2025-08-15 20:37:45.960707	5.0	
28978	u0651	p017	buy	2025-08-15 21:51:29.918447	2.0	
28992	u0714	p082	buy	2025-08-15 23:01:33.931232	5.0	
29008	u0287	p013	buy	2025-08-15 23:57:55.847335	5.0	
	quantity					
14	6.0					
21	5.0					
28	9.0					
37	7.0					
42	9.0					
...	...					
28963	10.0					
28968	10.0					
28978	3.0					
28992	3.0					
29008	10.0					

[2604 rows x 6 columns]

```
df_interactions.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29010 entries, 0 to 29009
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   user_id     29010 non-null  object
1   product_id  29010 non-null  object
2   event_type  29010 non-null  object
3   timestamp   29010 non-null  datetime64[ns]
4   rating      2604 non-null   float64
5   quantity    5847 non-null   float64

```

```
dtypes: datetime64[ns](1), float64(2), object(3)  
memory usage: 1.3+ MB
```

```
# convert df to csv  
  
df_interactions.to_csv('cannabis-user-data.csv', index = False)  
  
# download the file  
  
from google.colab import files  
  
files.download('cannabis-user-data.csv')
```